SpendLog AI

Architecture Report

System Design & Technical Architecture

Document No: DOC-01
Project: Agentic Expense Tracker (SpendLog AI)
Repository: https://github.com/Harish-S28/Agentic-expense-tracker
Prepared for: Project Documentation / README reference

1. System Overview

SpendLog AI is a full-stack, agentic personal expense tracker built on a monolithic Flask backend, a lightweight vanilla-JS frontend, a SQLite3 datastore, and a pluggable AI reasoning layer. The system follows a classic three-tier architecture (presentation, application/business logic, data), extended with an **agentic AI layer** that dynamically chooses between a Google Gemini large language model and a deterministic rule-based engine depending on API-key availability and runtime success.

The application is deployment-agnostic: it runs identically via **python app.py** on a developer machine or via **gunicorn** on Railway (or any nixpacks/Procfile-compatible PaaS), because database initialization (**init_db()**) is executed at module import time rather than only inside the __**main**__ guard.

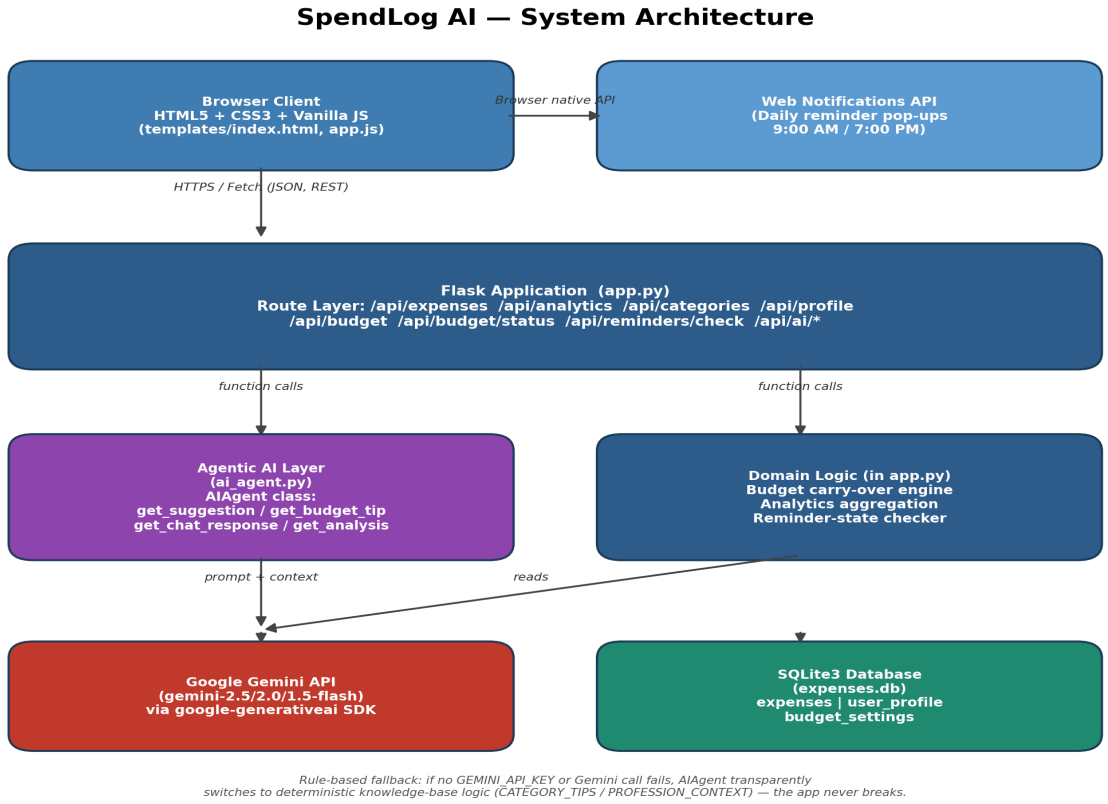

Figure 1. High-level layered architecture of SpendLog AI.

2. Architectural Layers

2.1 Presentation Layer

Located in **templates/index.html** and **static/js/app.js** / **static/css/style.css**. This is a single-page application shell: all views (Dashboard, Add Expense, Analytics, Budget, Profile/Persona, AI Chat drawer) are rendered client-side and communicate with the backend exclusively through a JSON REST API via the browser **fetch** API. The layer also owns two browser-native integrations: the **Web Notifications API** (for the twice-daily reminder alerts) and **setInterval**-based polling (checked every 60 seconds) to detect the 9:00–9:10 AM and 7:00–7:10 PM reminder windows.

2.2 Application / Routing Layer

Implemented entirely in **app.py** using Flask. Routes are grouped into: original CRUD/analytics endpoints (expenses, analytics, categories), and newer endpoints added for profile, budget, reminders, and AI features. Each route is a thin controller: it validates the JSON payload, opens a scoped SQLite connection via the **get_db()** context manager, executes parameterized SQL, and returns a JSON response.

2.3 Agentic AI Layer

Implemented in **ai_agent.py** as the **AIAgent** class. On initialization, if a Gemini API key is supplied, the agent probes a prioritized list of models (**gemini-2.5-flash** → **gemini-2.0-flash** → **gemini-1.5-flash** → **gemini-flash-latest**) with a minimal test call, and locks onto the first one that responds successfully. If no key is present, or every model probe fails, **has_gemini** stays False and the agent transparently falls back to a rule-based knowledge engine (**CATEGORY_TIPS** and **PROFESSION_CONTEXT** dictionaries) for every one of its four public methods: **get_suggestion**, **get_budget_tip**, **get_chat_response**, and **get_analysis**. This design gives the application graceful degradation — it is always able to respond, with or without external AI, and every Gemini call is wrapped in its own try/except that re-routes to the matching rule-based method on any runtime failure.

2.4 Data Layer

A single SQLite3 file, **expenses.db**, holds three tables: **expenses**, **user_profile** (a single-row table keyed at id=1), and **budget_settings** (one row per calendar month, uniquely keyed by the 'YYYY-MM' string). There is no ORM; all queries are handwritten parameterized SQL using Python's built-in **sqlite3** module with **row_factory = sqlite3.Row** for dict-like row access.

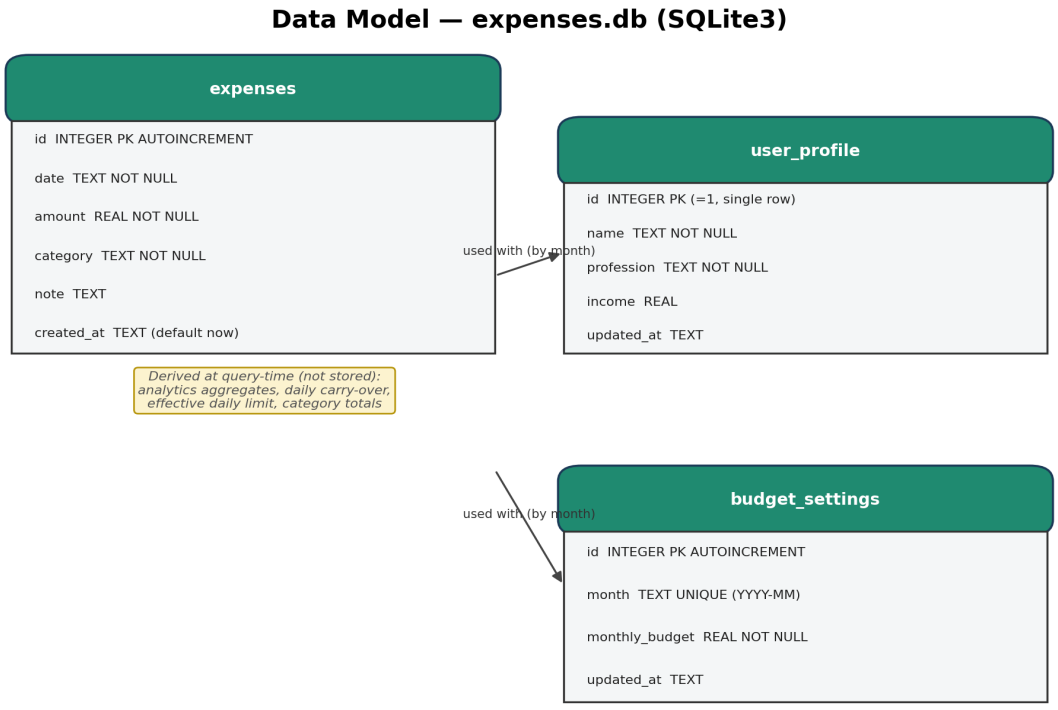

Figure 2. Logical data model of expenses.db.

3. Deployment Architecture

The repository ships a **Procfile** (declaring **web: gunicorn app:app**) and a **railway.toml** configuration, making it directly deployable to Railway using Nixpacks auto-detection. Locally, the same codebase runs via Flask's built-in development server. Configuration (chiefly **GEMINI_API_KEY** and **PORT**) is supplied through environment variables and loaded with **python-dotenv** in local development, or through the platform's native environment variable manager in production.

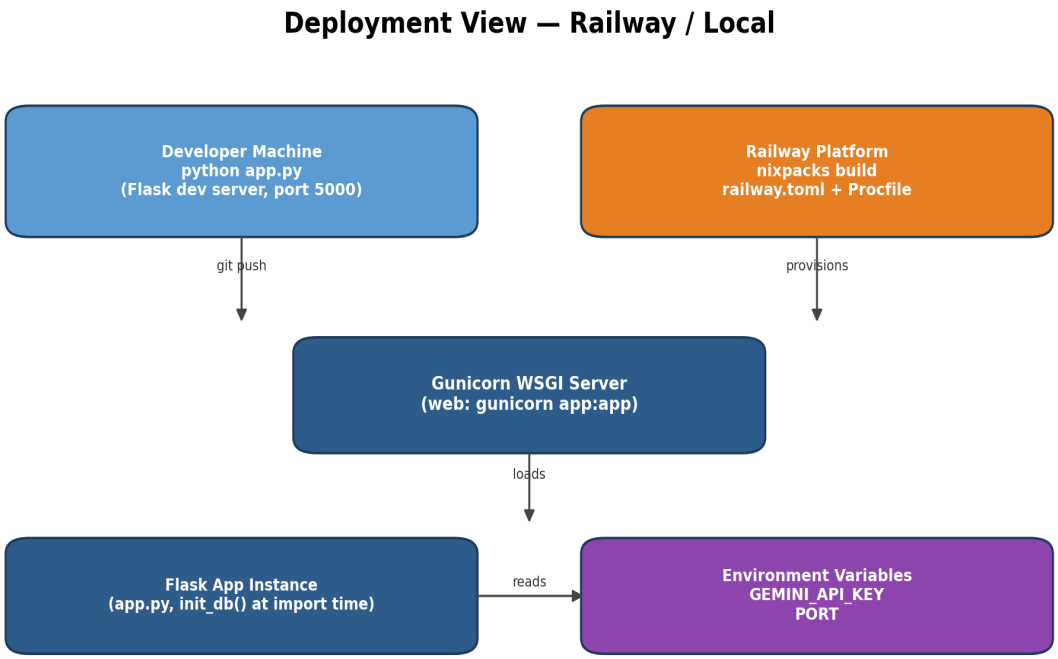

Figure 3. Deployment topology — local development vs. Railway production.

4. Technology Stack

Layer	Technology	Purpose
Backend framework	Flask 3.1.3	HTTP routing, request/response handling
WSGI server (prod)	Gunicorn 26.0.0	Production process manager on Railway
Database	SQLite3 (stdlib)	Embedded relational storage, zero external dependency
AI provider	google-generativeai 0.6.15 (Gemini 1.5/2.0/2.5 Flash)	LLM-driven suggestions, chat, analysis
Env management	python-dotenv 1.2.2	Loads .env for local API keys / config
Frontend	HTML5, Vanilla JS, CSS3	Single-page dashboard, no frontend framework/build step
Browser APIs	Fetch API, Notifications API	REST calls; native desktop-style reminder alerts
Deployment	Railway (Nixpacks) / Procfile	Cloud hosting with zero-config buildpacks

5. Request Lifecycle — Example

The sequence below traces a typical user action — logging an expense and receiving an AI-generated suggestion — through every architectural layer, illustrating how the presentation, application, data, and AI layers cooperate on a single user interaction.

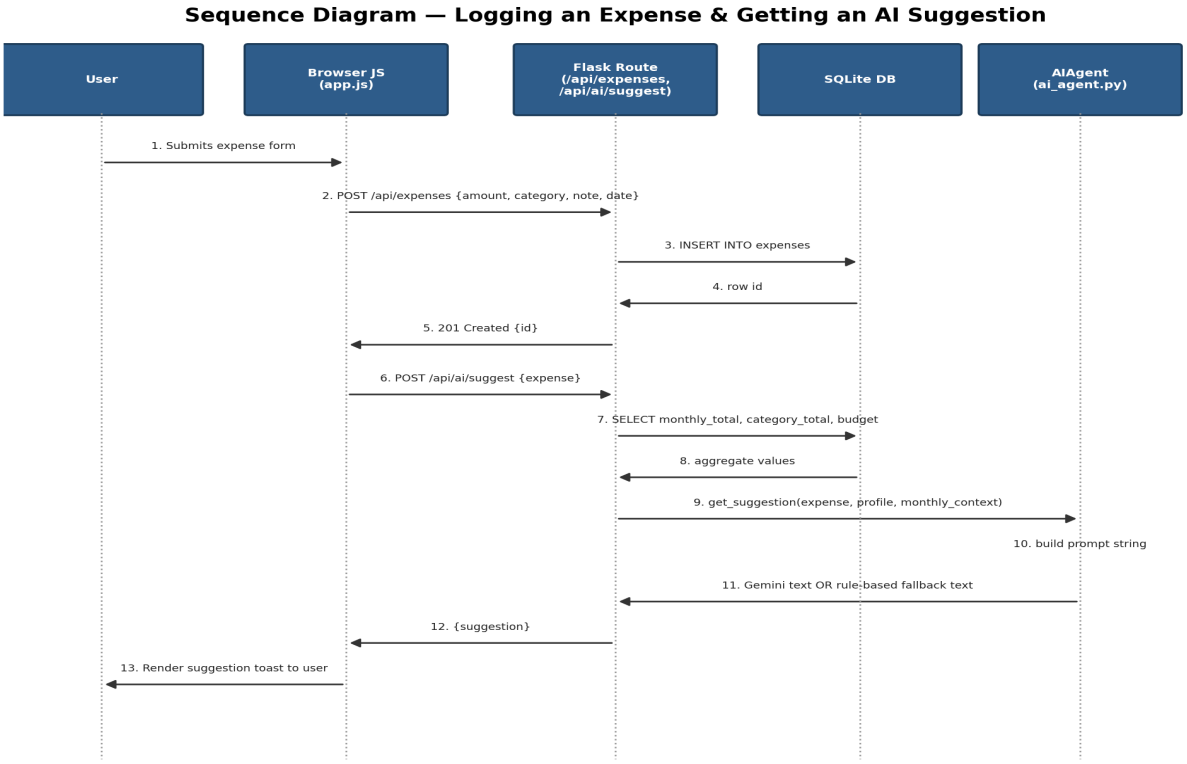

Figure 4. Sequence diagram for the 'add expense → AI suggestion' flow.

6. Key Architectural Decisions

- **Graceful AI degradation:** every AI-backed feature has a deterministic rule-based twin, so the product never breaks or blocks on external API availability.
- **Stateless routes, stateful database:** Flask routes hold no session state; all state (profile, budget, expenses) lives in SQLite, making the app trivially horizontally scalable behind a load balancer if needed.
- **Import-time initialization: init_db()** runs at module import rather than only under **if __name__=='__main__'**, which is what makes the Gunicorn-imported WSGI app self-provisioning in production without a separate migration step.
- **No ORM, explicit SQL:** keeps the data layer transparent and dependency-light, at the cost of manual query maintenance — an acceptable trade-off at this project's scale.
- **Model-priority fallback inside the AI layer itself:** the agent tries four Gemini model names in order of capability, so it self-heals against model deprecation or account-tier restrictions.